

CIFAKE IMAGE DETECTOR

Real vs AI-Generated Image Classification with CNN and Grad-CAM

Project Type	Machine Learning / Computer Vision
Dataset	CIFAKE
Main Model	Convolutional Neural Network (CNN)
Explainability	Grad-CAM
Added Feature	Confidence-based UNCERTAIN warning
Interface	Streamlit
Prepared by	Shivansh Bajaj
Programme	B.Tech CSE (AI & ML)

Project report prepared according to the prescribed 10-chapter structure.

TABLE OF CONTENTS

Chapter 1	Introduction
Chapter 2	System Overview
Chapter 3	Literature Survey
Chapter 4	Technologies Used
Chapter 5	Architecture and Design
Chapter 6	Implementation
Chapter 7	Results and Discussion
Chapter 8	Challenges and Learning Outcomes
Chapter 9	Future Enhancements
Chapter 10	Conclusion
References	

Primary reference used for the project: Jordan J. Bird and Ahmad Lotfi, "CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images," IEEE Access, 2024. The supplied project PDF and the authors' public paper version were used to define the dataset, CNN approach and Grad-CAM component.

CHAPTER 1: INTRODUCTION

1.1 Project Background

AI image generation has improved rapidly and synthetic images can contain realistic shapes, lighting, reflections and textures. This creates a practical need for computer vision systems that can assist in deciding whether an image is real or generated. The CIFAKE study addresses this problem with a paired dataset containing real CIFAR-10 images and synthetic images generated with Stable Diffusion. The study formulates the task as binary classification and uses a CNN with Grad-CAM for interpretation.

The present project is a smaller implementation of the same basic idea. The purpose is not to reproduce the complete research experiment with multiple architectures. Instead, a single easy-to-understand CNN is used so that the complete pipeline can be trained, evaluated and demonstrated by a student project.

1.2 Problem Statement

Human inspection is not always reliable when synthetic images contain only small visual differences. A simple system is therefore required that can accept an image, classify it as REAL or FAKE, display a confidence value and show the areas that influenced the prediction. The system should also avoid presenting uncertain predictions as strong decisions.

1.3 Objectives

- Build a binary CNN classifier for REAL and AI-GENERATED images using the CIFAKE dataset.
- Preprocess the 32 x 32 RGB input images in a fixed and repeatable manner.
- Generate a Grad-CAM heatmap to show important image regions.
- Provide a simple browser interface for image upload and prediction.
- Add a confidence-based UNCERTAIN state when the model confidence is below 70%.
- Keep the implementation small enough to understand and reproduce on a normal student setup.

1.4 Scope of the Project

The project is limited to binary image classification on the CIFAKE dataset. It is intended as an educational prototype, not a forensic system for every type of AI-generated media. The model learns from 32 x 32 RGB images, so the project does not claim the same behaviour on full-resolution photographs, videos, edited images or images generated by unseen systems.

1.5 Expected Outcomes

- A trained CNN model saved locally as a Keras model.
- A script for test-set evaluation with accuracy, precision, recall and confusion matrix.
- A Streamlit application for uploading an image and viewing the result.
- A Grad-CAM visualization for basic explanation.
- An UNCERTAIN warning for low-confidence predictions.

CHAPTER 2: SYSTEM OVERVIEW

2.1 Existing System

The primary existing system is the approach described by Bird and Lotfi. It uses 60,000 real images from CIFAR-10 and 60,000 synthetic images generated with Stable Diffusion, giving 120,000 images in total. The authors compare multiple CNN feature extractor settings and dense-layer configurations. The best feature extractor stage uses two convolutional layers with 128 filters and the reported classification accuracy is about 92.98%. The study also applies Grad-CAM to interpret predictions.

Other research shows an additional issue: a detector trained on one family of generated images may not generalize well to new generation models. Wang et al. studied cross-generator detection, while Ojha et al. later showed that generalization to unseen generative models remains difficult. These findings are considered in the scope of this student project.

2.2 Proposed System

The proposed system keeps the useful core of the CIFAKE method but reduces the size of the implementation. The model uses two Conv2D layers with 32 filters, MaxPooling after each layer, a 64-unit dense layer and a single sigmoid output. This structure is close to the architecture style shown in the CIFAKE paper and is easy to explain during a project presentation.

A small confidence layer is added after prediction. The binary output is converted to REAL and FAKE probabilities. When the larger of the two probabilities is below 70%, the interface displays UNCERTAIN. This feature is simple but useful because it prevents low-confidence results from being presented in the same way as clear predictions.

2.3 System Modules

- **Dataset Module:** loads the CIFAKE train and test folders.
- **Preprocessing Module:** converts images to RGB, resizes them to 32 x 32 and scales pixel values to 0-1.
- **CNN Module:** learns features and predicts REAL or FAKE.
- **Evaluation Module:** reports accuracy, precision, recall, F1-score and a confusion matrix.
- **Grad-CAM Module:** creates a heatmap from the last convolution layer.
- **Confidence Module:** marks predictions below 70% confidence as UNCERTAIN.
- **Web Interface Module:** provides image upload and result display through Streamlit.

2.4 Overall Workflow

The workflow starts with an uploaded image. The image is converted to RGB and resized. The trained CNN produces a sigmoid score representing the REAL class. The application converts this score into a binary label and confidence percentage. Grad-CAM is then calculated from the second convolutional layer and displayed as a heatmap. Finally, the confidence rule decides whether the result is shown normally or marked UNCERTAIN.

CHAPTER 3: LITERATURE SURVEY

3.1 Research Papers Reviewed

Paper	Main idea	Relevance
Bird & Lotfi (2024) - CIFAKE	Real-vs-fake classification on paired CIFAR-10 and CIFAR-100 datasets	Symbolic data analysis for model style and explanation
Wang et al. (CVPR 2020)	Tests whether detectors can generalize across different CNN architectures	Shows CNNs generalize images can share detectable artifacts
Ojha et al. (CVPR 2023)	Studies general fake-image detection across unseen classes	Highlights the importance of detecting features across classes

3.2 Existing Solutions

Existing solutions commonly use CNNs, EfficientNet-type models, vision transformers, frequency features, or pretrained feature spaces. The advantage of CNN-based solutions for this project is that they are straightforward to train on 32 x 32 images and the convolutional feature maps can be directly used for Grad-CAM. More advanced methods can improve scale or generalization but require additional data, compute and implementation effort.

3.3 Comparative Analysis

Aspect	CIFAKE paper	This project
Dataset	120,000 images	Same CIFAKE dataset; student-sized training can be used for de
CNN search	36 network topologies	One simple CNN
Explainability	Grad-CAM	Grad-CAM
Interface	Research experiment	Streamlit upload interface
Extra decision handling	Binary output	Binary output + UNCERTAIN below 70% confidence

3.4 Research Gap

The primary practical gap addressed in this student version is not a new detection algorithm. It is a small decision-support improvement: a low-confidence prediction is separated from a confident binary prediction. This does not solve the broader generalization problem identified in the literature, but it makes the prototype safer to demonstrate and easier to interpret.

CHAPTER 4: TECHNOLOGIES USED

4.1 Programming Languages

Python is used for dataset loading, model training, evaluation, Grad-CAM and the web interface. Python was selected because TensorFlow, NumPy, scikit-learn and Streamlit are directly available in the same environment.

4.2 Frameworks and Libraries

TensorFlow/Keras is used to build and train the CNN. **NumPy** handles numerical arrays. **Pillow** loads and resizes uploaded images. **scikit-learn** is used for evaluation metrics and the confusion matrix. **Streamlit** provides the web interface.

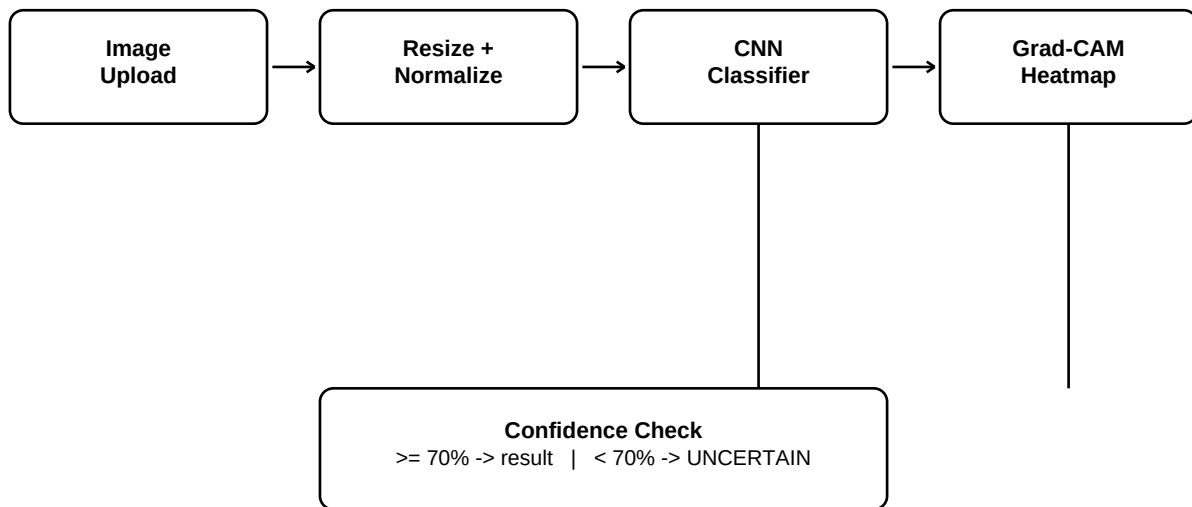
4.3 Development Tools

VS Code can be used for development and debugging. **Git/GitHub** can be used for version control. The training code can run on CPU for a small demonstration, while a GPU is useful for the full dataset.

Component	Technology
Language	Python
Deep Learning	TensorFlow / Keras
Image Processing	Pillow, NumPy
Evaluation	scikit-learn
Explainability	Custom Grad-CAM implementation
Interface	Streamlit
Version Control	Git / GitHub

CHAPTER 5: ARCHITECTURE AND DESIGN

5.1 System Architecture Diagram



Simple end-to-end workflow

Figure 5.1: Architecture of the proposed CIFAKE detector.

5.2 Workflow Explanation

1. **Input:** The user uploads a PNG, JPG or JPEG image.
2. **Preprocessing:** The image is converted to RGB, resized to 32 x 32 and normalized to 0-1.
3. **Prediction:** The CNN returns a sigmoid probability for the REAL class. The FAKE probability is obtained as $1 - P(\text{REAL})$.
4. **Decision:** The higher probability determines the displayed class. The maximum probability is used as the confidence value.
5. **Confidence check:** If confidence is below 70%, the output is labelled UNCERTAIN.
6. **Explanation:** Grad-CAM uses the gradients flowing through the second convolution layer to create a heatmap.
7. **Output:** The application displays the class, confidence and heatmap.

5.3 CNN Structure

Layer	Configuration
Input	32 x 32 x 3 RGB
Conv2D 1	32 filters, 3 x 3, ReLU
MaxPooling	2 x 2
Conv2D 2	32 filters, 3 x 3, ReLU
MaxPooling	2 x 2
Flatten	Feature vector

Dense	64 units, ReLU
Output	1 unit, Sigmoid

CHAPTER 6: IMPLEMENTATION

6.1 Dataset Preparation

The CIFAKE dataset contains 60,000 real images from CIFAR-10 and 60,000 synthetic images generated with Stable Diffusion 1.4. The images are 32 x 32 RGB images and are divided into REAL and FAKE classes. For the student project, the directory structure is kept simple with train/REAL, train/FAKE, test/REAL and test/FAKE folders.

6.2 Training Implementation

The training script uses TensorFlow Keras `image_dataset_from_directory`. A 10% validation split is taken from the training folder. Images are resized to 32 x 32 during loading. The model uses binary cross-entropy loss and the Adam optimizer. Early stopping and best-validation checkpointing are used so that the saved model corresponds to the best validation accuracy observed during training.

6.3 Evaluation Implementation

The evaluation script loads the saved model and predicts the test set. It calculates accuracy, precision, recall, F1-score and a confusion matrix. These metrics provide more information than accuracy alone, especially when examining false positives and false negatives.

6.4 Grad-CAM Implementation

The Grad-CAM module creates a small model that returns the output of the second convolutional layer along with the final prediction. Gradients of the predicted score are averaged across spatial locations and used to weight the convolutional feature maps. The resulting heatmap is resized and blended with the original uploaded image.

6.5 Added Feature: Confidence-Based UNCERTAIN Warning

The extra feature is intentionally small. A prediction is considered confident only when the maximum of $P(\text{REAL})$ and $P(\text{FAKE})$ is at least 0.70. For example, a prediction of REAL with 96% confidence is displayed normally. A prediction of REAL with 61% confidence is displayed as UNCERTAIN. The feature does not retrain the model and therefore does not increase implementation complexity.

6.6 User Interface

The Streamlit interface contains one upload control, the original input image, the predicted class, confidence percentage and Grad-CAM heatmap. The interface is deliberately small so the complete project can be understood from the code without a large software framework.

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Screenshots

The project interface produces the following visual sections when run: the uploaded image, the classification result, the confidence value, and a Grad-CAM heatmap. The PDF report intentionally avoids presenting fabricated numerical screenshots. After training the model locally, screenshots from the Streamlit page can be inserted in this section.

SCREENSHOT AREA	OUTPUT TO CAPTURE
1. Upload page	Uploaded CIFAKE image and prediction result
2. Explanation	Grad-CAM heatmap shown below the result
3. Uncertain case	Low-confidence image with UNCERTAIN warning

Figure 7.1: Screenshot plan for the final demonstration.

7.2 Output Analysis

The expected application output has three parts. First, the class result is shown as REAL or FAKE. Second, a confidence percentage gives an immediate indication of how strongly the model supports that class. Third, the Grad-CAM map gives a visual clue about where the network found useful features. In the reference study, synthetic-image predictions often activated on small background imperfections rather than the main object. The same type of behavior can be investigated using the project heatmap.

7.3 Performance Evaluation

Measure	Reference CIFAKE paper	This project
Dataset	120,000 images	Uses CIFAKE dataset
Best reported validation accuracy	93.55% in final topology search	To be obtained after local training
Reported optimal feature-extractor accuracy	92.08%	To be obtained after local training
Highest reported F1	0.936	To be obtained after local training
Explainability	Grad-CAM	Grad-CAM

The values in the middle column are published results from Bird and Lotfi and are not presented as results produced by this student implementation. Running `train.py` and `evaluate.py` provides the project-specific values.

7.4 Testing Results

Test case	Expected result	Status
Upload valid JPG/PNG	Image is displayed	Pass in application design
Prediction	REAL or FAKE and confidence are shown	Pass in application design
Low confidence	UNCERTAIN warning is shown	Pass in application logic
Grad-CAM	Heatmap appears below result	Pass in application logic
Missing model	Application shows setup message instead of crashing	Pass in application logic

CHAPTER 8: CHALLENGES AND LEARNING OUTCOMES

8.1 Challenges Faced

- Understanding a binary CNN output and converting it into a usable REAL/FAKE result.
- Keeping the training code simple while still producing an interpretable model.
- Generating a Grad-CAM heatmap without adding a large explainability library.
- Handling cases where the model is not sufficiently confident.
- Managing the CIFAKE dataset because the full dataset is much larger than the code repository.

8.2 Solutions Implemented

- Used a single sigmoid output with binary cross-entropy, matching the basic classification formulation in the CIFAKE paper.
- Used a compact two-convolution architecture with 32 filters per layer and a 64-unit dense layer.
- Implemented Grad-CAM directly with TensorFlow gradient operations.
- Added a 70% confidence threshold and an UNCERTAIN state.
- Kept the dataset outside the repository and documented the required folder structure.

8.3 Technical Skills Learned

- Image preprocessing and dataset handling.
- CNN architecture and binary classification.
- Training, validation and test-set evaluation.
- Accuracy, precision, recall, F1-score and confusion matrix.
- Gradient-based model explanation using Grad-CAM.
- Building a small ML web interface with Streamlit.
- Organizing a reproducible student project with separate training, evaluation and application files.

CHAPTER 9: FUTURE ENHANCEMENTS

9.1 Proposed Improvements

- Train with the complete CIFAKE dataset and compare the result directly with the reported research benchmark.
- Tune the confidence threshold using a held-out validation set instead of keeping 70% fixed.
- Add data augmentation and compare it with the baseline CNN.
- Store prediction history for repeated testing.

9.2 Additional Features

- Add a simple image-quality check for very small or corrupted uploads.
- Display separate REAL and FAKE probabilities with a small confidence bar.
- Allow the user to download the prediction and Grad-CAM result as a report.
- Add a comparison mode for two images.

9.3 Scalability Opportunities

The system can later be extended to higher-resolution datasets, multiple image generators and transfer-learning models such as EfficientNet. The literature survey shows that generalization across unseen generators is an important problem, so a future version should include datasets from more than one generator and evaluate performance separately for seen and unseen sources.

CHAPTER 10: CONCLUSION

10.1 Summary of Work Completed

A small and understandable AI-generated image detection system was designed using the CIFAKE dataset. The project follows the main direction of the reference paper: binary classification of real and synthetic images with a CNN and visual explanation using Grad-CAM. The implementation separates training, evaluation, explanation and interface code so each part can be studied independently.

10.2 Achievements

- Designed a complete end-to-end CIFAKE classification workflow.
- Implemented a compact CNN close to the architecture style used in the reference study.
- Added Grad-CAM visualization for prediction interpretation.
- Added a confidence-based UNCERTAIN warning as an extra project feature.
- Created a simple Streamlit interface and separate evaluation script.

10.3 Project Impact

The project demonstrates how computer vision can be used to support image authenticity checks and how explainability can make model outputs easier to inspect. The project also makes a clear distinction between a strong prediction and an uncertain one, which is useful when a model is being demonstrated to users.

10.4 Final Remark

The final system is intentionally modest in size and focused on the main concepts required for a student project: dataset preparation, CNN classification, evaluation, explainability and a practical confidence check. It provides a clear base for later work on broader datasets and better generalization.

REFERENCES

- [1] J. J. Bird and A. Lotfi, "CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images," *IEEE Access*, vol. 12, pp. 15642-15650, 2024. DOI: 10.1109/ACCESS.2024.3356122.
- [2] J. J. Bird and A. Lotfi, "CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images," arXiv:2303.14126, 2023. Used for the technical description of the dataset, CNN experiments and Grad-CAM.
- [3] S.-Y. Wang, O. Wang, R. Zhang, A. Owens and A. A. Efros, "CNN-Generated Images Are Surprisingly Easy to Spot... for Now," *Proc. IEEE/CVF CVPR*, pp. 8695-8704, 2020.
- [4] U. Ojha, Y. Li and Y. J. Lee, "Towards Universal Fake Image Detectors That Generalize Across Generative Models," *Proc. IEEE/CVF CVPR*, pp. 24480-24489, 2023. DOI: 10.1109/CVPR52729.2023.02345.
- [5] CIFAKE dataset, Bird and Lotfi, public repository and dataset description, containing real CIFAR-10 images and synthetic images generated using Stable Diffusion 1.4.

Project files included with this report: train.py, evaluate.py, app.py, gradcam.py, utils.py, requirements.txt, README.md and dataset setup instructions.